Software Process Discovery Using Link Analysis

Beichen Duan
School of Software
Shanghai Jiaotong University
Shanghai, China
duanbc@gmail.com

Beijun Shen
School of Software
Shanghai Jiaotong University
Shanghai, China
bjshen@sjtu.edu.cn

***Abstract*—Data mining is an exhausting yet very useful task. It can help programmers understand the nature of development process and reveal potential problems during the process. Thus more reliable software can be created. As open source software development communities thrive, many software repositories with useful information about the development process become publicly available, enabling us to do research. In this paper, the author explores an approach based on mapping the artifacts to the process activity of FreeBSD repository to study the control-flow of the development process.**

I. INTRODUCTION

In principle, software development process offers a framework to ensure reliable and predictable result when creating and modifying software system. There are a lot of well-known process models, such as Waterfall Model [5], Spiral Model [6], V-model [7], and all kinds of agile software development processes [8].We are uncertain about whether practitioners comply with the guideline or have interest in how software development proceeds. With techniques from mining software repository, we can gain insight of software development process from the corresponding repository.

Software development process can provide a great deal of useful information which can be classified into several aspects: control flow aspect, information aspect as well as organizational aspect.

Recently, Open source software development has drawn the attention both of academia and of industry [1]. Mining software repository (MSR) community has done tremendous work [2-4] to help manage and improve the progress of software projects through mining the information provided by the software repositories such as source control system, archived communications between project personnel, and defect tracking systems, etc.

This research explores a framework to semi-automatically restore the control-flow of the process from the repository. This paper begins with a brief review of what others have achieved (Section 2). A description of how the framework works (section 3) follows. Then the experiment alone with its result is presented (section4). The author ends this paper with potential threats (section 5) and the conclusion of this research (section 6).

II. RELATED WORK

Metaprocess, such as Capability Maturity Model (CMM) [9], attempts to model the processes used to create software, much like ISO standard 9000 [10] attempts to model and document how processes are executed, tracked, modeled and documented. The CMM concerns modeling, tracking, and ranking software process adherence.

Laura Maruster et al. [11] present a development method of a model that estimates the probability of direct successor relation of two events A and B. W.M.P. van der Aalst et al. [12] introduce the topic of process mining. They illustrate not only the potential of process mining but also many scientific challenges that need to be addressed. Additionally, they present both a language and a tool to enable the verification of properties based on event logs [13]. B.F.van Dongen et al. [14] introduce a mining tool ProM which integrates the functionality of several existing tools and provides many additional process mining plug-ins. They all focus more on the generalized process (including business process, work flow process, etc.) rather than the software development process. Some unique attributes of the development process are not fully explored.

Chris Jensen .et al. [15] describe a framework for software process discovery in open source software development communities that provide software process context. Their framework can help point the direction. Additionally, they present an approach to discovering software processes, combining techniques for text analysis, repository usage and update pattern [16]. However, the potential benefits of some techniques needs to be dug out.

Abram Hindle et al. [17] present high-level views about the software processes of a project through integrating many mining software repository technologies and comparing them with the rational unified process. However, more details are required in order for us to gain more insight of software development.

III. APPROACH

Our approach consists of three stages.

First of all, the repository has to be obtained. Then an event log file can be generated from the repository. Many source control systems including CVS, Subversion, SourceSafe, Clear Case, support this feature. In order to be independent from a particular SCM system, a common format for the event log is defined. The versioning log consists of records (rows). Records are produced after checkins of the documents during different executions of different software processes. Each record should contain at least three attribute of the action: documents, timestamp, author, .etc. The whole framework works primarily on the set of records that belong to one process.

At the next stage of the approach, the link between file and process activity is defined. A meaningful model of the

978-1-61284-486-2/11/$26.00 ©2011 IEEE

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:59:02 UTC from IEEE Xplore. Restrictions apply.

development process consists of process activities which can be distinguished through their own properties, such as actions, tools, resource agents, as well as artifacts they produces. The author chooses artifacts to differentiate the activities with. In other words, the evidence can be seen as reference to process activities. Here, the author assumes that all logs have the same naming conventions and that a kind of activity would result in specific artifacts. E.g. the document containing the test-specific artifacts is called "test". Document whose name contains "test" would be the result of some kind of activity involved with testing.

Then activities are sorted by the timestamp. The activities after sorting construct the control aspect of a process. There are many languages to describe software processes. Compared to process program language, object-oriented method and visualization method, though complex Petri Net is formal and supports the dynamic evolution of process. So, Petri Net is chosen to describe the software process in this paper. Different algorithms which derive control-flow mining have been proposed. It is not the essential goal of this paper to describe the algorithms.

The approach is presented in Fig.1.

IV. EXPERIMENT

FreeBSD is a free Unix-like operating system and FreeBSD is released under a variety of open source licenses. Repository of FreeBSD is publicly available, making it a good dataset on which to do process mining. Thanks to the MSR community, a full svn mirror of freebsd (to date September 16th 2009) can be obtained [18]. Four subprojects of freeBSD are chosen for the experiment: arpr2_merge_1, jbuild, vap7, and mips.

We use "svn log" utility provided by Subversion to generate logs of the repository. Since the log contains too many details and very specific document names and paths, which are irrelevant to the process mining, we carry on some manipulations on it. An example of the log file after manipulations is shown in Table 1.

Then a mapping between the evidence and the process activity is determined. In our algorithm for text mining, the type and the absolute path of files are taken as inputs and the files are classified into categories of artifacts of the corresponding process activities. 5 kinds of process activities are assumed to exist in the FreeBSD repository and the links are as follows. Following the FreeBSD conventions, all the committed documents containing "/src/" in their paths or having ".h" ".hh" ".java", ".cc" or ".c" as an extension are mapped to "CONSTRUCT" activity, all the documents named "README" are mapped to "README" activity, all the files containing "test" in their paths or names are mapped to "TEST" activity, all the "Makefile.*" files to "BUILDE" activity, and all the files, whose names end with ".conf" or contain "conf" to "CONFIG" activity. All the other commits are ignored.

After all process activities are sorted, represented by the corresponding artifacts, Petri Net as shown in Fig.2 is obtained. For the sake of readability, a simplified Petri Net without loops which was obtained by killing the loops is shown in the paper.

TABLE I. PART OF EVENT LOG

Document	Timestamp	Author
arpr2_merge_1/bin /cat/cat.c	08.10.30 22:05:57	Ivoras
arpr2_merge_1/bin /cat/Makefile	01.12.04 9:57:47	Obien
arpr2_merge_1/bin /test/test.c	05.01.10 16:39:26	Imp
……		
build/release/doc/ Makefile	07.08.17 22:16:59	Bmah
jbuild/tools/test/RE ADME	08.05.17 10:09:48	Jb
jbuild/bin/test/test. c	09.05.28 10:18:58	dwmalono

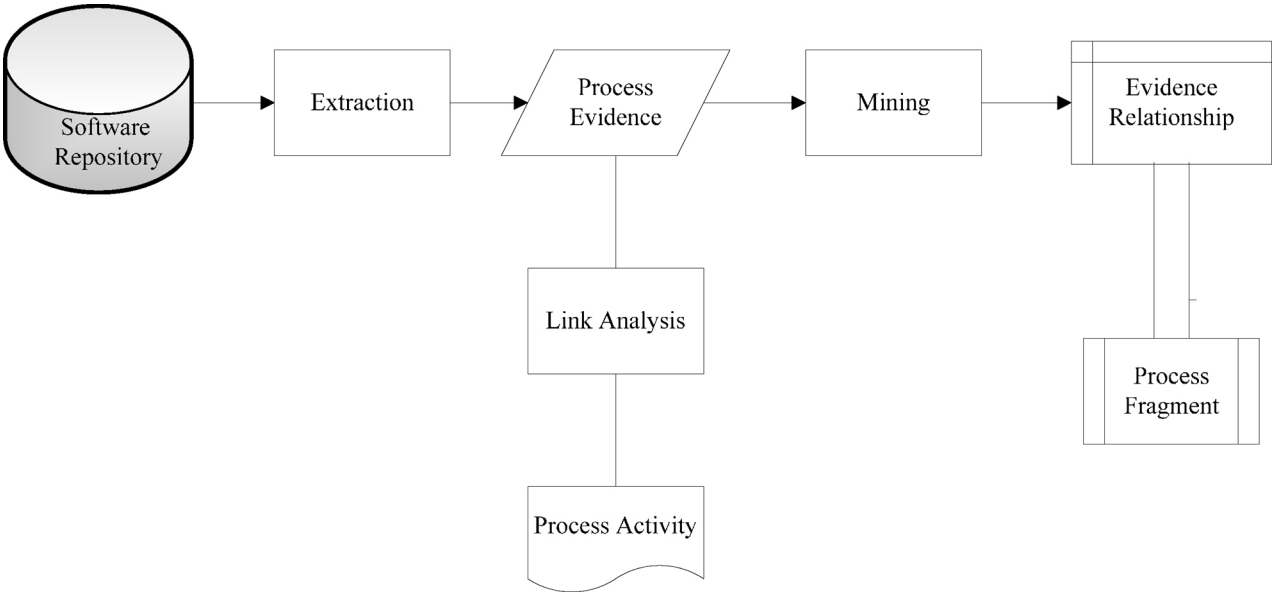
From the model, we can clearly draw the conclusion that people start with building the MAKEFILE or editing the README file, then they write code to implement the required functions, after that, they test and configure it, or the other way around. It's worth noting sometimes README file is changed after writing code. Additionally, we can notice some code would directly go to application without testing, which indicates that more attention should be paid to quality assurance (QA).

V. POTENTIAL THREADS

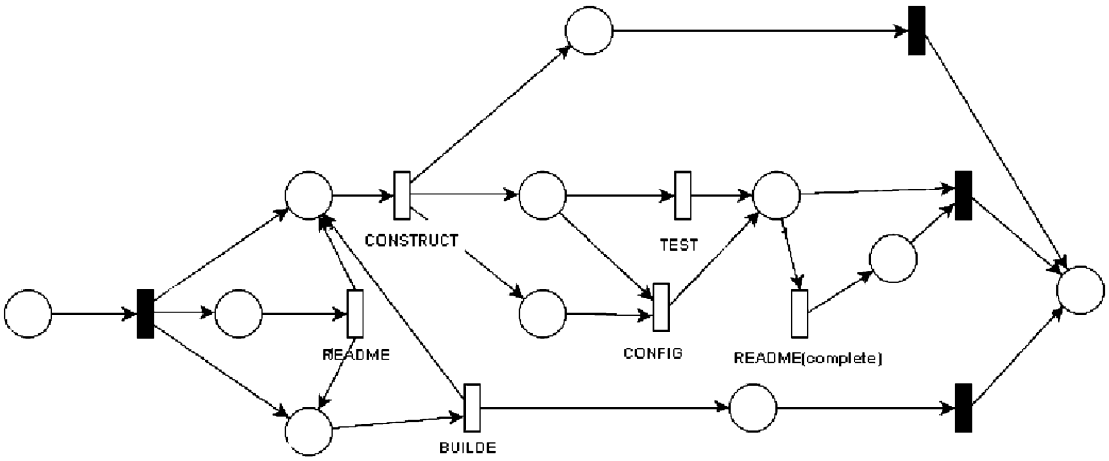
The framework presented in this paper is novel and flexible. However, there are some disadvantages which would pose threats on it.

Firstly, this research is laid on the output and artifacts left behind by programmers, which varies among projects in terms of amount of details and degree of

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:59:02 UTC from IEEE Xplore. Restrictions apply.



Figue.1. Process mining framework using link analysis



Figue.2. Petri Net for the experiment

consistence. Some projects are more consistent in their practices, while others are not so consistent.

Secondly, it's dangerous to extend the link of artifacts and process activities used above, for it can be different among projects. For example, it's easier to determine the artifacts of CONSTRUCT for the projects with Java than those with C++. The accuracy of the mapping can affect the results greatly. While applying this framework to some specific projects, we need to determine the link by following the convention of project.

Thirdly, we must consider that the development artifacts may not be available or observable through the repository. For example, project requirements may be managed by a group external to the development team, and key but undocumented design decisions may be made informally by developers in face-to-face meeting. Some projects store their design or requirement in other systems which are not open publicly. So the discovery of process activity is limited to what we can observe or what the communities provide.

VI. CONCLUSION

In this paper, the author has proposed a framework for discovering software development process on the repositories of projects, especially of the open source

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:59:02 UTC from IEEE Xplore. Restrictions apply.

software projects. This author studies the process in which FreeBSD is developed and gains some insight from the experiment. The experiment is carried on FreeBSD, but we can see that it is a framework that can easily be tailored to other field as long as analogous information can be provided. However, the disadvantages of the framework would still need much work to be done in the future.

ACKNOWLEDGMENT

This research is supported by the Opening Project of Key Laboratory of High Confidence Software Technologies (No. HCST201001). Also the authors would like to extend their thanks to Yangming Lei for his invaluable help on this paper.

REFERENCES

[1] Christian Bird, Brendan Murphy, Nachi Nagappan, and Thomas Zimmermann. 2011. Empirical Software Engineering at Microsoft Research, in Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW2011), Association for Computing Machinery

[2] J. Ekanayake, J. Tappolet, H. C. Gall, and A. Bernstein. 2009. Tracking Concept Drift of Software Projects Using Defect Prediction Quality, in IEEE Working Conference on Mining Software Repositories, Vancouver, Canada

[3] Thomas Ball, 2009, A Brief History of Software - from Bell Labs to Microsoft Research, Keynote at Working Conference on Mining Software Repositories(2009), Vancouver, Canada

[4] Carlo Ghezzi. 2008. Dynamically Evolving Software: Some Radical Changes of Perspective, Keynote at Working Conference on Mining Software Repositories(2008), Leipzig, Germany

[5] Boehm B. 1986. A Spiral Model of Software Development and Enhancement, ACM SIGSOFT Software Engineering Notes", "ACM", 11(4):14-24, August

[6] Rook, Paul, E. Rook. 1986. Controlling software projects, IEEE Software Engineering Journal, 1(1), (1986), pp. 7-16.

[7] Larman, Craig. 2004. Agile and Iterative Development: A Manager's Guide. Addison-Wesley. P.27 ISBN 9780131111554

[8] V. Rubin, C.W. Günther, W.M.P. van der Aalst, E. Kindler, B.F. van Dongen, and W. Schäfer. 2007. Process Mining Framework for Software Processes. International Conference on Software Process, Software Process Dynamics and Agility (ICSP 2007), Q. Wang, D. Pfahl, and D.M. Raffo

[9] 1997 Capability maturity model for software, Tech. Rep. CMU/SEI-91-TR-24 ADA 240603.

[10] D. Stelzer, W. Mellis, and G. Herzwurm. 1997. A critical look at iso 9000 for software quality management, Software Quality Control, vol. 6, no. 2, pp. 65–79.

[11] L. Maruster, A.J.M.M. Weijters, W.M.P. van der Aalst, and A. van den Bosch. Process Mining. 2002. Discovering Direct Successors in Process Logs. Proc. 5th International Conference on Discovery Science, (2002) Berlin.

[12] W.M.P. van der Aalst and A.J.M.M. Weijters. 2004. Process mining: A Research Agenda, Computers in Industry, (2004) 53(3):231-244

[13] W.M.P. van der Aalst, H.T. de Beer, and B.F. van Dongen. 2005. Process Mining and Verification of Properties: An Approach based on Temporal Logic. BETA Working Paper Series, (2005) WP 136, Eindhoven University of Technology, Eindhoven

[14] van Dongen, B., Medeiros, A., Verbeek, H., Weijters, A., van der Aalst, W. 2005. The ProM framework: A New Era in Process Mining Tool Support. In Ciardo, G., Darondeau, P., eds.: Application and Theory of Petri Nets (2005). Volume 3536.(2005) 444–454

[15] Jensen, C. and Scacchi, W. 2003. Applying a Reference Framework to Open Source Software Process Discovery, 2003, Proc. 1st Workshop on Open Source in an Industrial Context, OOPSLA-OSIC03, (Anaheim, CA, October 2003)

[16] Jensen, C. and Scacchi, W. 2004. Data Mining for Software Process Discovery in Open Source Software Development Communities, Proc. 1st Workshop on Mining Software Repositories, ICSE04-MSR04, (Edinburgh, Scotland, May 2004)

[17] Abram Hindle, Michael W. Godfrey and Richard C. Holt. 2010. Software Process Recovery using Recovered Unified Process View. Proc. of the 2010 IEEE Intl. Conference on Software Maintenance (ICSM-10), Timisoara, Romania,

[18] http://msr.uwaterloo.ca/msr2010/challenge/

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:59:02 UTC from IEEE Xplore. Restrictions apply.